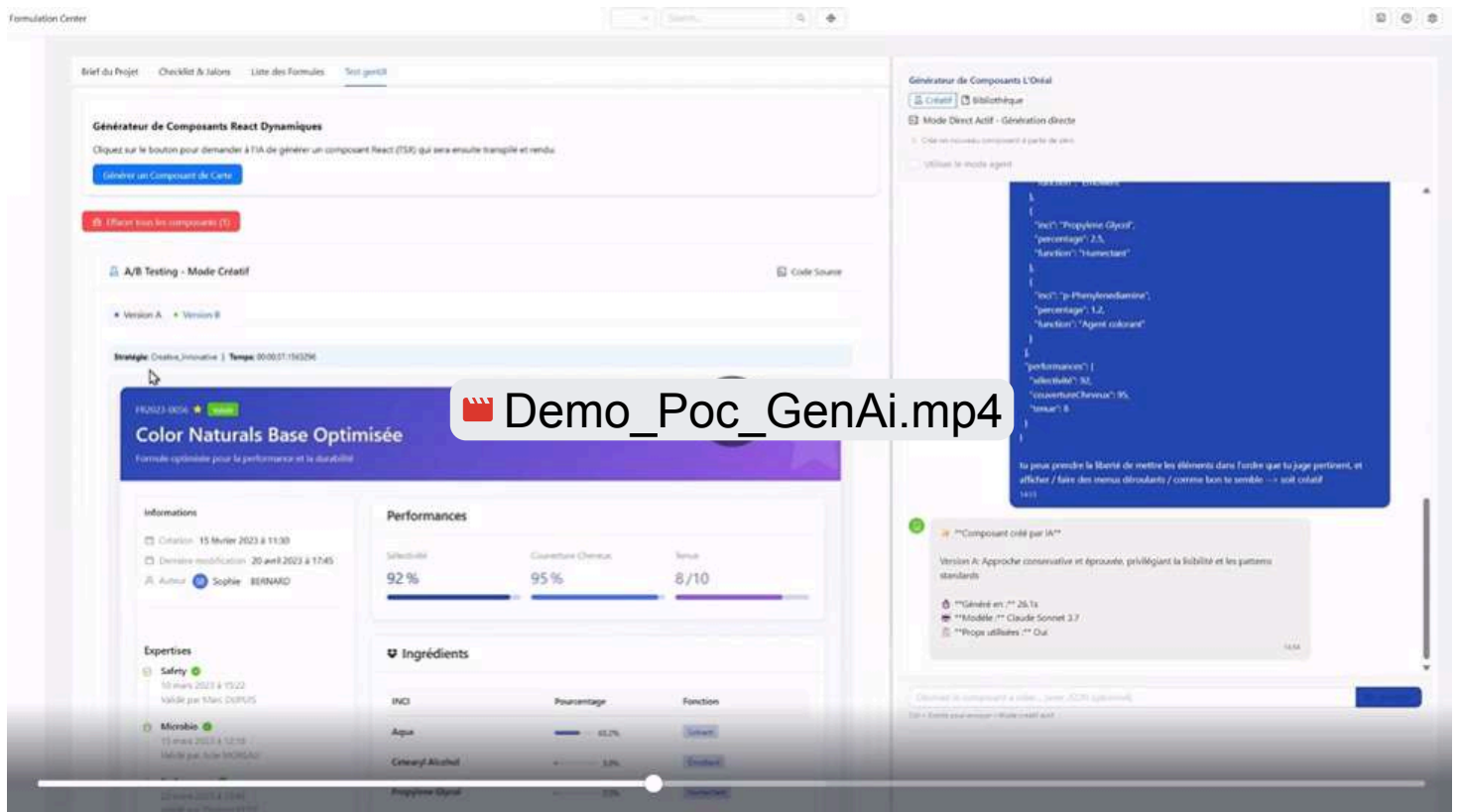


Portfolio réalisations IA

Productions pour L'Oréal.....	2
PoC GenAI : générateur de composants pour application métier.....	2
Outil de visualisation de dépendance entre applications.....	3
Chaîne agentique de gestion de contexte projet.....	4
Productions universitaires.....	5
Revue de littérature automatisée en partenariat avec Thales :.....	5
Benchmark des implémentations d'un diagramme de voronoï.....	6
Application mobile de recommandation de voyage (mode avion).....	8
Production bonus.....	9
Nuit de l'info 2026, visualiseur audio Analify.....	9

PoC GenAI : générateur de composants pour application métier



J'ai mené ce PoC en deux mois, pendant mon stage de fin de BUT2 (équivalent bac+2) pour L'Oréal. Au sein des équipes de la plateforme qui regroupe les logiciels à destinations des formulateurs et des chimiste, il y avait un besoin d'accélération du time du delivery avec un goulot d'étranglement au niveau du design. C'est pourquoi ce PoC a vu le jour, cherchant à identifier les forces de la GenAI sur la partie design d'interface pour répondre à des besoins métiers de plus en plus spécifiques. En générant des composant directement en React TypeScript, dans le respect du design system L'Oréal et avec la possibilité d'avoir les données réelles, cette preuve de concept fut un succès et mon travail à été industrialisé dès la fin de mon stage

Outil de visualisation de dépendance entre applications

Lors de mon deuxième stage chez L'Oréal pour ma fin de BUT (bac+3) je dispose de six mois pour comprendre les pain points des différentes équipes, collaborer pour consolider leurs besoins et concevoir des solutions adaptées. Dans ce cadre j'ai été positionner sur un sujet d'augmentation de la gouvernance par l'IA. En effet, parmi les différents besoins des équipes, la question des dépendances techniques (API, data quality dans les référentiels) est souvent revenue. Je suis donc chargé d'un projet de cartographie des différentes dépendances entre applications. Cela passe par les données des différents appels API, les données du référentiel (APM ServiceNow), et les données des codes sources qu'il faut explorer et croiser. J'étudie donc des agents de rétro documentation du code source, des outils de cartographie, et des moyens de requêter ces outils via une question en langage naturel (via un LLM).



Graphe des dépendances entre API et applications - R&I

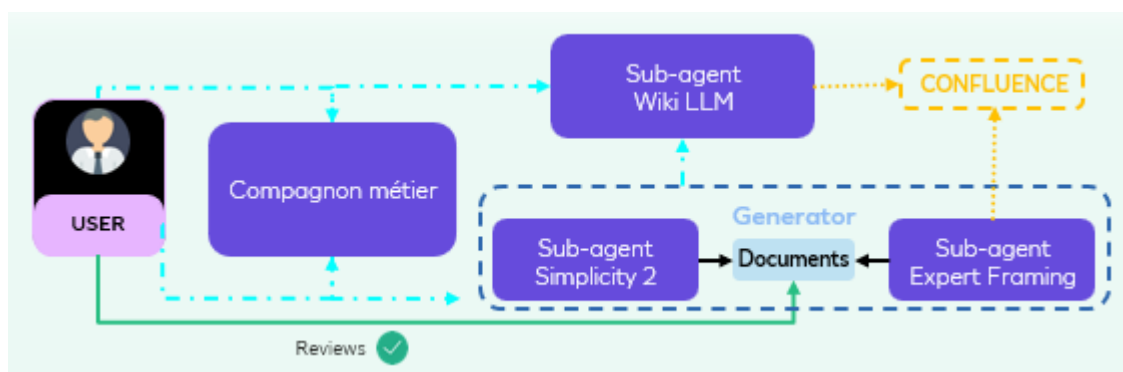
Cet outil de visualisation des dépendances permettra de donner facilement un contexte transverse à un LLM et d'optimiser le fonctionnement d'un autre projet : le Wiki LLM.

Chaîne agentique de gestion de contexte projet

Avec une volonté d'outiller la gouvernance des projets grâce à l'IA et en renfort de l'initiative "SDLC agentique"¹, j'étudie l'efficacité d'un système de wiki LLM. Popularisé par Andrej Karpathy (co-fondateur d'open AI), c'est une méthode qui permet d'ingérer des documents dans un wiki en les découpant en unités sémantiques reliées entre elles, pour bénéficier des avantages du graph-RAG.

J'ai donc conçu un "compagnon" L'Oréal GPT (agent avec système prompt, outils, et mcp confluence) orchestrateur pour créer les pages du wiki (dans confluence) et faire l'ingestion de nouveaux documents.

Il est possible de faire ingérer tout type de contenu textuel, mais pour aller plus loin l'idée est de faire générer des documents types par d'autre agent IA, puis de construire une chaîne agentique pour alimenter le wiki en continue.



vision cible : chaîne agentique de gestion de contexte projet

Un agent génère des comptes rendus de réunion à partir des transcriptions, des messages de conversation et du support de présentation (en cours de test).

Un autre est chargé de la génération des documents de framing (vision, persona, user journey, kvi, epic, roadmap, us) pouvant alimenter la base de connaissance des contextes projets.

Ces différents projets sont aujourd'hui éprouvés au travers d'une migration qui touche de nombreuses applications et demande des analyses d'impacts et l'élaboration de planning capacitaires. C'est pourquoi tous ces outils sont voués à être regroupés dans une plateforme pour optimiser l'écosystème de travail d'acteur comme les architectes, les BRM ou les équipes data.



Hub d'accueil de la plateforme qui regroupe les projets auxquels je participe

¹ Software Delivery Life Cycle, chaîne agentique avec MCP figma make, ServiceNow, Confluence, GitHub... pour une accélération de la mise en production d'un logiciel.

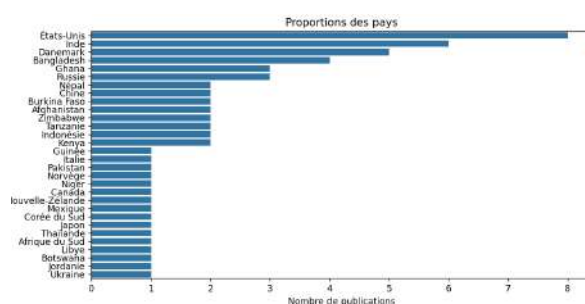
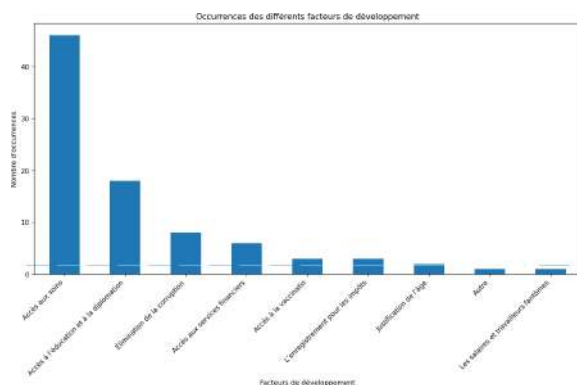
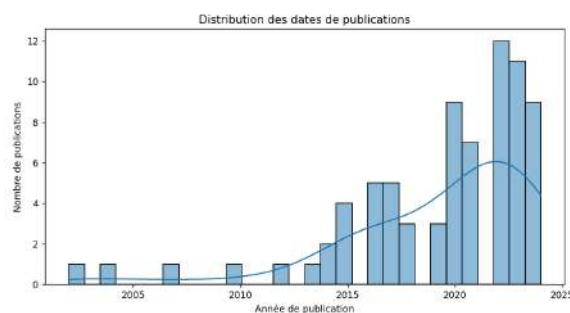
Productions universitaires

Revue de littérature automatisée en partenariat avec Thales :

Ce projet, proposé dans le cadre du cours « Management des Systèmes d'Information » en partenariat avec THALES, vise à apporter des éléments de réponses à la question « les systèmes d'identité influencent-ils certains facteurs de développement dans les pays en voie de développement ».

L'objectif est ainsi d'obtenir comme produit fini un tableau récapitulatif, exploitable pour les personnes souhaitant étudier cette question, en prenant en considération des études publiées sur le sujet, avec les informations les plus pertinentes et une analyse intégrée, reposant en partie sur l'Intelligence Artificielle.

Nous avons donc appris à utiliser l'API de google scholar pour obtenir des informations sur une centaine d'articles scientifiques pour ensuite faire des appels à l'API Groq (contraintes d'une API gratuite). En plus du tableau récapitulatif nous avons fourni plusieurs graphiques donnant une vision d'ensemble sur le corpus étudié (pandas, matplotlib et streamlit).



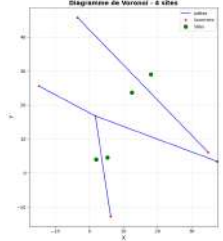
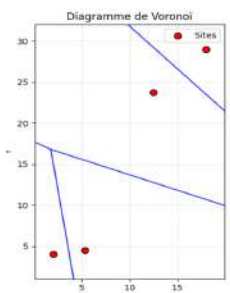
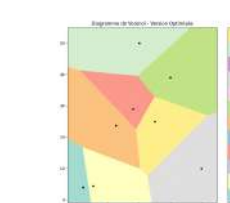
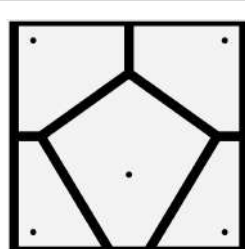
```
load_dotenv()

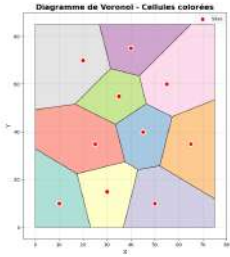
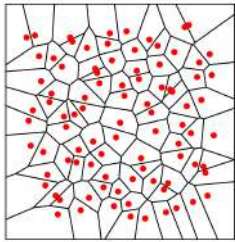
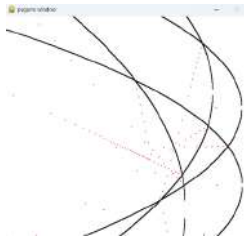
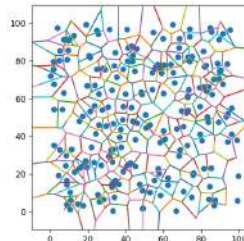
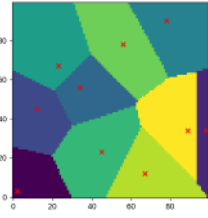
client = Groq(
    api_key=os.getenv("API_KEY"),
)

def chat_with_groq(prompt):
    try:
        r = client.chat.completions.create(
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            temperature=0.2,
            model="llama3-70b-8192",
        )
        # Extraire et retourner la réponse du modèle
        return r.choices[0].message.content
```

extrait du code python d'appel à l'API Grog

Benchmark des implémentations d'un diagramme de voronoï

Nom du LLM	Le résultat répond au cahier des charges ?	Temps	Problèmes rencontrés	Résultat du diagramme de voronoï	Mesure de la performance
Claude sonnet 4.5	Pratiquement, excepté le calcul du nombre de sommet et l'aspect "joli"	~1h	Mauvaise implémentation de l'algorithme de fortune, et prolongement des segments ouvert en fin d'algorithme		$O(n^3)$ et 11,2 secondes en moyenne pour 200 points
Grok 4.1	Oui sauf ajout d'une fonctionnalité pour mesurer les performances en fonction du nombre de points chargés sur l'interface tkinter	47 mins 31	- Tests qui ne passent pas - Affichage mauvais en raison d'une échelle du diagramme trop grande		$O(n^3)$ 100 points : ~0,9s 200 points : ~4,5s 500 points : ~57s
Mistral IA Le Chat	Cahier des charges très partiellement rempli (uniquement la représentation des points)	50 min 45 s	- Mauvais chemins d'accès aux modules proposés par le modèle. - algorithme implémenté dans un script mais non fonctionnel - code qui détecte bien les points mais ne réalise pas de diagramme.		non mesurée
Gemini	Pratiquement bon avec l'algorithme de balayage point par point	~ 1h15	problème d'environnement et de version avec python, d'optimisation des calculs, conflit avec l'interface		$O(W*H*N)$ où W et H la hauteur et largeur de la grille 2.42 secondes pour 100 points et 23 secondes pour 1000 points
Perplexity	Oui, entièrement. Implémentation complète de l'algorithme par intersection de demi-plans, export SVG fonctionnel, CLI opérationnelle, suite de tests fournie.	30–40 minutes	Erreur de parsing du pyproject.toml (BOM/encodage). Installation éditable nécessitant correction.		$O(n^3)$ 100 points: 0.025 s 200 points: 0.106 s 500 points: 0.769 s

Deepseek	Cahier des charges remplis	~2h	- Assez long pour envoyer le rendu de chaque fichier (+10 min) -création de fichiers / dossiers qui restent vides et inutilisés -nombreuses erreurs et oublis, comme utiliser une bibliothèque dans le code mais il oublie de l'importer ou bien il confond x et y dans une fonction		$O(n^3)$ 100 points: 0.02527s 200 points: 0.10570s 500 points: 0.76888s
ChatGPT	Le résultat est rempli avec un algorithme naïf simple, bien que l'esthétique soit rudimentaire	~ 10 minutes	Résultat produit instantanément, un fichier de configuration manquant qu'il a fallu demander puis redemander corrigé. Les points n'étaient pas affichés au départ seulement les traits et l'affichage était trop petit.		$O(n^3)$ 4 points : 0.000158 secondes 100 points : 0.083414 secondes 500 points : 6.319317 secondes
Développement humain - Ioan	Résultat satisfaisant pour un premier jet avec une sweepline et le calcul de toutes les intersections entre les paraboles grâce à une modélisation géométrique complète	~ 10h	Pas de beachline donc détection d'intersections inutiles et diagramme final incomplet, c'est l'étape 1 de la réalisation de l'algorithme de fortune		
Développement humain - Lucas	si ce n'est la batterie de tests unitaire et le prolongement des segments le résultat est satisfaisant	15~20h	Algorithme simple à comprendre mais difficile à implémenter.		$O(n \cdot \sqrt{n} \cdot \log n)$ 200 points en 2s.
Développement humain - Razi, Lorenzo, Vincent, Léona, Salma	En terme d'optimisation et respect des bonnes pratiques du code, c'est faible, mais le résultat et l'affichage sont bons	6h	Problème avec matplotlib pour l'affichage, on a donc utilisé imshow() pour simplifier le coloriage		$O(W \cdot H \cdot N^2)$ où W est la largeur, H la hauteur en pixels, et N le nombre de germes

Ce travail en groupe de sept effectué en BUT 3 avait pour objectif de comparer et discuter les écarts entre une implémentation 100% étudiante et une 100% assistée par l'IA en se demandant où se situe le juste milieu. Conclusion : 1) un temps incompressible est nécessaire pour apprendre et espérer être ensuite efficace avec un LLM. 2) Tous les modèles ne se valent pas pour cette tâche précise.

Application mobile de recommandation de voyage (mode avion)

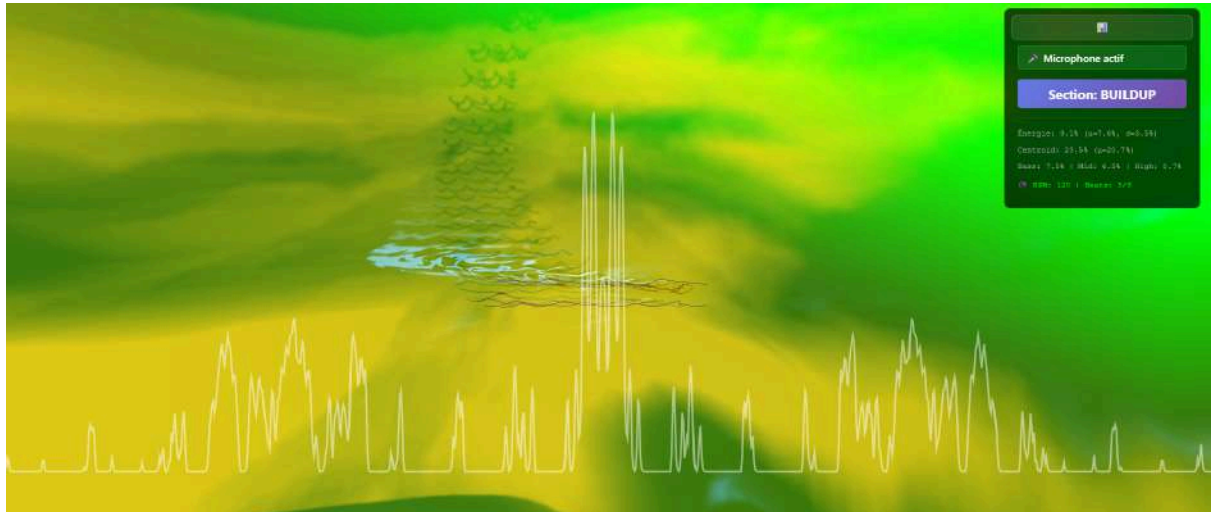
Dans ce projet l'IA nous a aidé à collecter les données pour les embarquer en base SQLite



Avec une équipe de sept étudiants nous avons développé une application de recommandation de voyage. Nous avons réparti le travail en trois compétences : le développement front-end, la qualité des données, l'optimisation algorithmique. La majeure contrainte était de faire une application fonctionnelle hors ligne, donc nous avons fonctionné avec une base SQLite comportant des données sur les destinations, des activités, et les prix moyens des billets d'avions. Nous avons collecté ces données en nettoyant des dataset Kaggle et en faisant des scripts en python d'appel vers des API (LLM pour les vols, et API classiques pour les activités et les images). L'interface est réalisée avec Flutter et pour ce qui est de la recommandation nous avons choisi de travailler en triant des scores de similarité cosinus entre un vecteur utilisateur et un vecteur destination à six composantes. Pour un peu plus de sérendipité nous avons ajouté la possibilité de muter le vecteur utilisateur sur une composante aléatoire de temps en temps (10% des résultats). Enfin un effet de mode à court terme vient pondérer fortement le vecteur utilisateur en fonction des like ou dislikes qu'il peut émettre dans un mini-jeu lui proposant cinq destinations. Notre goulot d'étranglement ne fut pas l'algorithme de recommandation mais les requêtes SQLite. Nous sommes toutefois parvenue à donner vingt destinations en 0.3s sur un smartphone récent.

Production bonus

Nuit de l'info 2026, visualiseur audio [Analify](#)



Ce projet, réalisé en une nuit en trinôme, m'a beaucoup amusé et replongé avec nostalgie dans l'époque où j'étais fasciné par les écrans de windows media player.

Notre objectif était de faire un visualiseur open source et intelligent. Nous avons donc utilisé une bibliothèque open source pour l'analyse audio et avons configuré des modèles de visualisations audio grâce. Nous avons utilisé l'Intelligence Artificielle générative pour l'interface et ajouter une fonctionnalité qui permet de visualiser un mp3 uploadé dans le site mais aussi le son ambiant du micro de l'ordinateur ou de tout autre appareil.

